Retrieval-Augmented Generation (RAG)

A Conceptual Overview

> **Lecture Type:** General Overview — suitable for students, non-technical stakeholders, and developers new to the topic.

---

1. The Problem: AI Models Don't Know Everything

Imagine hiring a brilliant consultant who read every textbook, research paper, and article published up to a certain date — but then was locked in a room with no internet, no documents, and no updates. That's essentially what a large language model (LLM) is.

LLMs are trained on massive amounts of text, but that training has a **cutoff date**. More importantly, they have no knowledge of:

- Your company's internal documents
- Your product specifications and pricing
- A user's personal history or preferences
- Events that happened after their training ended
- Your organization's private policies and procedures

When asked about these things, models don't say "I don't know." They often **hallucinate** — they generate confident-sounding answers that are simply wrong.

The core problem RAG solves:

> *How do we give an AI model the right information, at the right time, to answer a specific question accurately?*

---

2. The Naive Solutions (and Why They Fall Short)

Before RAG, there were two obvious approaches. Both have serious limitations.

Option A: Retrain or fine-tune the model on your data

You could take a base model and continue training it on your company's documents. The model would "memorize" your data. But this is expensive, slow, requires machine learning

expertise, and the moment your data changes — a policy update, a new product — you'd need to retrain again.

Option B: Stuff everything into the prompt

Modern LLMs have large "context windows" — they can read thousands or even hundreds of thousands of words in a single prompt. You could just paste all your documents into every query.

This works for very small knowledge bases. But it breaks down quickly:

- Most real knowledge bases contain millions of documents — far too large for any context window
- Longer contexts cost more money (you pay per token with most APIs)
- Models actually perform *worse* with extremely long contexts — they lose focus and miss details buried in the middle
- Not all information is relevant to every query — why load a printer manual when someone is asking about your return policy?

---

3. The RAG Solution: Retrieve, Then Generate

RAG takes a fundamentally different approach. Instead of giving the model *everything*, you give it only what it needs for *this specific question*.

The name says it all:

- **Retrieval** — find the most relevant pieces of information from your knowledge base
- **Augmented** — add that information to the model's context
- **Generation** — let the model generate an answer using that context

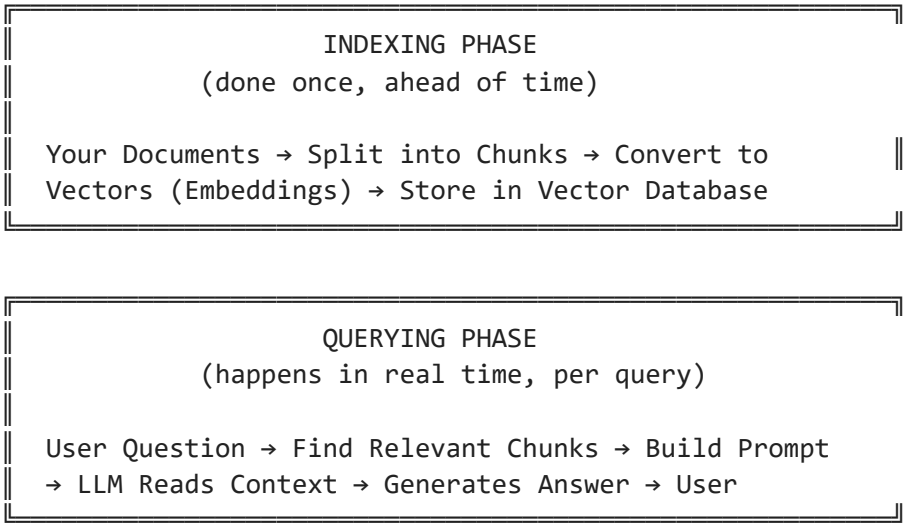
Think of it like an **open-book exam**. A closed-book exam forces you to memorize everything in advance. An open-book exam lets you look up exactly what you need. Students who can find the right passage and synthesize it well do better than those trying to recall facts from memory.

The model is the student. Your knowledge base is the book. RAG is the ability to open it.

---

4. How RAG Works — The Big Picture

A RAG system has two main phases: **indexing** (done once, in advance) and **querying** (done in real time, for every user question).



Let's walk through each step in plain language.

Step 1: Chunking — Breaking Documents Into Pieces

Your documents could be hundreds of pages long. Retrieving an entire document for a single question is wasteful and noisy. Instead, you break each document into small, manageable **chunks** — think of them as individual paragraphs or short sections.

Each chunk should be small enough to be precise, but large enough to contain meaningful context. A typical chunk might be 300–500 words.

Step 2: Indexing — Making Chunks Searchable

Once chunked, each piece of text is processed so it can be quickly found later. There are different ways to do this, which we'll cover in Section 6. The result is an **index** — a data structure optimized for fast lookup.

Step 3: Retrieval — Finding What's Relevant

When a user asks a question, the system searches the index for the chunks most relevant to that question. It might retrieve 2, 5, or 10 chunks — just the most useful ones.

Step 4: Augmentation — Building the Prompt

The retrieved chunks are combined with the user's question into a single prompt. The model is instructed to answer based only on the provided context.

Step 5: Generation — The Model Answers

The LLM reads the question and the retrieved context, then generates a grounded, accurate response. Because the answer is based on real retrieved text, hallucinations are dramatically

reduced.

---

5. A Concrete Example

Let's say you're building **KenteCodeGPT** — a customer support chatbot for KenteCode AI Academy, an online academy that offers cohort-based AI courses. Your knowledge base contains course catalogs, pricing details, and academy policies.

User asks: "*What is the cost of the Gen AI class?*"

Without RAG, the model might guess — or confidently give a wrong price.

With RAG:

1. The system searches the knowledge base for chunks about the Generative AI Engineering course
2. It retrieves the course details: "*The Generative AI Engineering program is a 6-month advanced course priced at $2,000. A 50% discount is available for residents of Africa.*"
3. That chunk is added to the prompt as context
4. The model reads it and answers: "*The Generative AI Engineering course costs* $2,000. If you're a resident of Africa, you qualify for a 50 1,000.*"

The model didn't "know" this. It read it, just like you would.

---

6. How Retrieval Works — Three Approaches

The retriever is the heart of a RAG system. Getting retrieval right is the biggest factor in RAG quality. There are three main approaches, each with its own strengths.

Approach 1: Term-Based Retrieval (Keyword Search)

This is the oldest approach — the same idea behind traditional search engines. The system looks for documents that contain the same words as the query.

How it works: Each document is indexed by the words it contains. When you search for “printer speed A300”, the system finds documents that contain those words. More sophisticated versions (like BM25) weight rare words more heavily than common ones — “A300” is more distinctive than “the”.

Strengths: Very fast. Works great for exact matches — product names, model numbers, error codes, unique identifiers.

Weakness: It can't understand meaning. A query for "how to get my money back" won't match a document about "refund policy" unless those exact words overlap.

Approach 2: Semantic (Embedding-Based) Retrieval

This is the modern approach, powered by neural networks. Instead of matching words, it matches *meaning*.

Every chunk of text is converted into a list of numbers called an **embedding** — a mathematical representation of the text's meaning. Similar meanings produce similar numbers. The query is also converted to an embedding, and the system finds the chunks whose embeddings are closest.

How it works: Think of meaning as a map. "Return policy", "refund", "get my money back", and "how to return an item" all live close together on this map. The retriever finds whatever is nearby, regardless of exact wording.

Strengths: Understands synonyms, paraphrases, and intent. Much more flexible and natural.

Weakness: Can be slower and more expensive. Can also miss exact technical terms (like error code `EADDRNOTAVAIL`) because they get "averaged out" in the embedding.

Approach 3: Hybrid Search

Production RAG systems almost always combine both approaches — using keyword search to catch exact matches and semantic search to catch conceptual matches. The results are merged using a technique called **Reciprocal Rank Fusion**, which combines the two ranked lists into a single best-of-both result.

This is the recommended approach for real applications.

---

7. A Peek at the Code — What This Looks Like in Practice

You don't need to understand every line below. The goal is to show that the concepts we've discussed map directly to simple, readable code. Deeper dives into each component will come in later lectures.

Code Block 7.1 — Install Required Libraries

In [ ]:
```
# These are the main tools we'll use throughout the RAG series
# !pip install openai sentence-transformers faiss-cpu rank_bm25
```

Code Block 7.2 — KenteCode AI Academy Knowledge Base

In [1]:
```
# KenteCode AI Academy knowledge base — real course and policy data
# Source: https://www.kentecode.ai/
documents = [
    {
        "id": "doc_001",
        "title": "AI for Everyone Course",
        "content": (
            "AI for Everyone is a 4-week beginner-friendly course priced at $199. "
            "Students learn practical AI skills with ChatGPT, Claude, Perplexity AI
            "and NotebookLM to work smarter. No technical background is required. "
            "Enrollment is currently open."
        )
    },
    {
        "id": "doc_002",
        "title": "Generative AI Engineering Course",
        "content": (
            "The Generative AI Engineering program is a 6-month advanced course pri
            "at $2,000. Students go from Python basics to building and deploying "
            "production-grade AI systems. The curriculum covers LLMs, RAG, agents,
            "vector databases, and includes a hands-on capstone project. "
            "A 50% discount is available for residents of Africa."
        )
    },
    {
        "id": "doc_003",
        "title": "AI Automation Course",
        "content": (
            "The AI Automation course is a 12-week intermediate-level program price
            "at $799. Students learn to build automated workflows using n8n and Zap
            "to streamline operations. This course is coming soon."
        )
    },
    {
        "id": "doc_004",
        "title": "Academy Overview and Policies",
        "content": (
            "KenteCode AI Academy is an online cohort-based learning platform based
            "Houston, TX. Founded by Vincent Amedekah, a Staff Data Engineer with 1
            "of experience. The academy offers live weekend sessions with industry
            "personalized instructor-led mentorship, and hands-on portfolio project
            "The community has over 1,000 members. A 50% discount is available for
            "residents across all courses. Contact: info@kentecode.ai."
        )
    },
]

print(f"KenteCode AI knowledge base ready: {len(documents)} documents loaded.")
for doc in documents:
    print(f"  [{doc['id']}] {doc['title']}")
```

```
KenteCode AI knowledge base ready: 4 documents loaded.
  [doc_001] AI for Everyone Course
  [doc_002] Generative AI Engineering Course
  [doc_003] AI Automation Course
  [doc_004] Academy Overview and Policies
```

Code Block 7.3 — Seeing Retrieval in Action (BM25)

In [2]:
```python
from rank_bm25 import BM25Okapi

# Step 1: Tokenize each document (split into words)
tokenized_docs = [doc["content"].lower().split() for doc in documents]

# Step 2: Build the BM25 index
bm25 = BM25Okapi(tokenized_docs)

# Step 3: Ask a question and retrieve the most relevant document
query = "What is the cost of the Gen AI class?"
tokenized_query = query.lower().split()
scores = bm25.get_scores(tokenized_query)

# Show which document scored highest
best_match_idx = scores.argmax()
print(f"Query: '{query}'")
print(f"Best match: [{documents[best_match_idx]['title']}]")
print(f"Content: {documents[best_match_idx]['content']}")
```

```
Query: 'What is the cost of the Gen AI class?'
Best match: [Generative AI Engineering Course]
Content: The Generative AI Engineering program is a 6-month advanced course priced a
t $2,000. Students go from Python basics to building and deploying production-grade
AI systems. The curriculum covers LLMs, RAG, agents, vector databases, and includes
a hands-on capstone project. A 50% discount is available for residents of Africa.
```

Code Block 7.4 — The Full KenteCodeGPT Prompt (What the LLM Actually Sees)

In [3]:
```python
# This is the core of KenteCodeGPT — combining retrieved context with the user's
# question into a single prompt that grounds the model's answer in real information

retrieved_chunk = documents[1]["content"]  # The Gen AI Engineering course (retriev
user_question = "What is the cost of the Gen AI class?"

rag_prompt = f"""You are KenteCodeGPT, a helpful assistant for KenteCode AI Academy
Answer the user's question using ONLY the context provided below.
If the answer is not in the context, say so.

CONTEXT:
{retrieved_chunk}

USER QUESTION:
{user_question}

ANSWER:"""
```

```
print("=== The prompt sent to the LLM ===")
print()
print(rag_prompt)
```

```
=== The prompt sent to the LLM ===

You are KenteCodeGPT, a helpful assistant for KenteCode AI Academy.
Answer the user's question using ONLY the context provided below.
If the answer is not in the context, say so.

CONTEXT:
The Generative AI Engineering program is a 6-month advanced course priced at $2,000.
Students go from Python basics to building and deploying production-grade AI system
s. The curriculum covers LLMs, RAG, agents, vector databases, and includes a hands-o
n capstone project. A 50% discount is available for residents of Africa.

USER QUESTION:
What is the cost of the Gen AI class?

ANSWER:
```

In [4]:
```
# With an OpenAI API key, you would send this prompt like so:
import os
from dotenv import load_dotenv
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
from openai import OpenAI
client = OpenAI(api_key=api_key)
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": rag_prompt}]
)
print(response.choices[0].message.content)
```

```
The cost of the Generative AI Engineering program is $2,000.
```

---

8. When Should You Use RAG?

RAG is powerful, but it's not the right tool for every situation. Here's a simple guide.

RAG is a great fit when:

- Your application needs to answer questions about specific documents, policies, or records
- Your knowledge base changes frequently (new products, updated policies, recent events)
- You need the model to cite its sources or stay grounded in verified information
- Your knowledge base is too large to fit in any context window
- You're building customer support, document Q&A, internal search, or research tools

RAG may not be necessary when:

- Your entire knowledge base is under ~200,000 tokens (roughly 500 pages) — in that case, you can simply include everything in the prompt
- You need general knowledge or reasoning, not specific document lookup
- Your data never changes and is small enough to fine-tune on

A note on context length: Some people argue that as context windows grow larger, RAG will become obsolete. This is unlikely. Data grows faster than context windows, and stuffing irrelevant information into every prompt increases cost and reduces model focus. RAG solves a structural problem, not just a size problem.

---

9. Common Real-World Use Cases

RAG is now one of the most widely deployed patterns in production AI systems. Here are examples you've likely encountered or could build:

Use Case	What's Retrieved	What's Generated
Customer support chatbot	Product manuals, FAQs, policies	Accurate answers grounded in docs
Internal knowledge base	Company wikis, HR policies, SOPs	Answers for employees
Legal document assistant	Contracts, case law, regulations	Clause summaries, risk flags
Medical information tool	Clinical guidelines, drug references	Patient-facing explanations
News research assistant	Recent articles and reports	Summaries with source attribution
Code documentation helper	API docs, code comments	Usage examples and explanations
Course Q&A bot	Lecture notes, textbook chapters	Student question answers

---

10. What Makes a RAG System Good or Bad?

The quality of a RAG system comes down to two things: **did you retrieve the right chunks?** and **did the model use them well?**

Retrieval quality is measured by:

- **Precision** — of the chunks retrieved, how many were actually relevant?

- **Recall** — of all the relevant chunks in your database, how many did you find?

A retriever that finds irrelevant chunks wastes the model's context. A retriever that misses the right chunk means the model has nothing useful to work with.

Generation quality is measured by:

- **Faithfulness** — did the model stick to what was in the retrieved context, or did it make things up?
- **Relevance** — did the answer actually address what the user asked?

The biggest mistake teams make is assuming that better LLMs fix bad retrieval. They don't. If the retriever returns the wrong chunks, even the best model will give a wrong or hallucinated answer. **Retrieval quality is the foundation.**

---

11. What's Coming Next

This lecture gave you the conceptual foundation for RAG. In the upcoming deep-dive lectures, we'll go hands-on with each component:

Lecture	Topic
RAG Part 2	Chunking strategies — fixed-size, sentence-based, recursive, overlapping
RAG Part 3	Embeddings — what they are, how to generate them, how to choose a model
RAG Part 4	Vector databases — FAISS, Pinecone, ChromaDB, MongoDB Atlas
RAG Part 5	Hybrid search — combining BM25 and semantic retrieval with RRF
RAG Part 6	Query rewriting and multi-turn conversations
RAG Part 7	Contextual retrieval and chunk augmentation
RAG Part 8	Evaluating RAG systems end-to-end with RAGAS

By the end of the series, you'll be able to build a production-ready RAG system from scratch.

---

12. Key Terms to Remember

Term	Plain-Language Definition
RAG	A pattern where an AI retrieves relevant documents before generating an answer
Chunk	A small piece of a document — the unit that gets retrieved

Term	Plain-Language Definition
Embedding	A list of numbers that represents the meaning of a piece of text
Vector Database	A database that stores embeddings and supports fast similarity search
Retriever	The component that finds relevant chunks given a user's question
Generator	The LLM that reads the retrieved chunks and produces an answer
Context Window	The maximum amount of text an LLM can read in one prompt
Hallucination	When a model generates confident but factually incorrect information
BM25	A classic keyword-based retrieval algorithm (term-based)
Semantic Search	Retrieval based on meaning rather than exact word matching
Hybrid Search	Combining keyword and semantic retrieval for best results
Precision	Fraction of retrieved chunks that are relevant
Recall	Fraction of all relevant chunks that were retrieved